

Groceries

The Groceries problem illustrates the two most important **routing** problems in Operations Research: The Traveling Salesman Problem (TSP) and the Vehicle Routing Problem (VRP). Both problems have been studied intensely over the last 50 years and have a large number of practical applications, like parcel deliveries, technician routing or pickup of waste and recycling. Both TSP and VRP are computationally hard optimization problems, even for small problem sizes. Here we focus on relatively simple models. If larger problems need to be solved, more advanced algorithmic approaches are needed instead of general solvers.

Problem 1

- Minimize the travel distance of the van.

Sets

- Set of locations, including both customers and the grocery store as the first "customer": $c \in Customers = \{1, 2, 3, 4, 5, 6\}$

Parameters

- $Coord_{c,ind}$: Coordinates of customer c used for calculating the Distance Matrix. The index is such that $Coord_{c,1}$ holds the x -coordinate and $Coord_{c,2}$ the y -coordinate.
- $Distance_{c1,c2}$: The (Euclidean) distance between customer $c1$ and $c2$, calculated based on the coordinates.

Decision variables

- Drive van from customer $c1$ to $c2$: $x_{c1,c2} \in \{0,1\}$. 1 reflects "yes" and 0 reflects "no". Notice that the variables $x_{c,c}$, i.e. driving from node c to node c itself, does not make any sense. There are several options for handling these. One can exclude them from the model by only defining the

variables for $c1 \neq c2$ and making similar restrictions in all summations. Alternatively, as done here, one can define the variables but fix but them to zero.

Model

Objective:

- Minimize the driven distance:

$$\sum_{c1,c2} Distance_{c1,c2} \cdot x_{c1,c2}$$

Constraints:

- You have to drive **to** each customer exactly once:

$$\sum_{c2} x_{c2,c1} = 1 \quad \forall c1$$

- You have to drive **away** from each customer exactly once:

$$\sum_{c2} x_{c1,c2} = 1 \quad \forall c1$$

- Only define the variables that make sense:

$$x_{c1,c1} = 0 \quad \forall c1$$

The above model is very simple and fast to solve, but has a serious issue (Problem 2 below).

The full model in Julia/JuMP, available with the name

`Groceries1.jl`

from the book web-site, is given below:

```
#####
# Groceries1, "Mathematical Programming Modelling" (42112)
#####
# Intro definitions
using JuMP
using HiGHS
#####
```

```

#####
# PARAMETERS
C=6 # no of customers

#x and y- coordinates of all customers, c=0 is the store:
coord = zeros(C,2)
coord[1,1]=0;   coord[1,2]=0
coord[2,1]=104; coord[2,2]=19
coord[3,1]=370; coord[3,2]=305
coord[4,1]=651; coord[4,2]=221
coord[5,1]=112; coord[5,2]=121
coord[6,1]=134; coord[6,2]=515

Distance = zeros(Float64,C,C)
for c1 in 1:C
    for c2 in 1:C
        Distance[c1,c2]= sqrt( (coord[c1,1]-coord[c2,1])^2 + (coord[c1,2]-coord[c2,2])^2 )
    end
end
#####

#####
# Model
TSP = Model(HiGHS.Optimizer)

@variable(TSP, x[1:C,1:C], Bin)
for c in 1:C
    fix(x[c,c],0; force = true) #"Remove" nonsense variables by setting to 0
end

# Minimize TSP distance
@objective(TSP, Min,
    sum(Distance[c1,c2]*x[c1,c2] for c1=1:C, c2=1:C))

# Arrive at each customer ONCE:
@constraint(TSP, [c1=1:C],
    sum(x[c2,c1] for c2=1:C)==1)

# Leave each customer ONCE:
@constraint(TSP, [c1=1:C],
    sum(x[c1,c2] for c2=1:C)==1)
#####

#####
# solve

```

```

optimize!(TSP)
println("Termination status: $(termination_status(TSP))")
#####

#####
println("objective = $(objective_value(TSP))")
println("Solve time = $(solve_time(TSP))")
for c1=1:C
    for c2=1:C
        if value(x[c1,c2])>0.01
            println("Van drives from $c1 -> $c2")
        end
    end
end
end
#####

```

If you look at the solution, it is clear that the solution consists of two separate loops: $c1 \rightarrow c2 \rightarrow c5 \rightarrow c1$ and $c3 \rightarrow c6 \rightarrow c4 \rightarrow c3$. The problem is that these routes actually satisfy the constraints, but constraints are missing which will disallow multiple tours (sub tour elimination). What is needed is the addition of a new variable and one constraint.

Problem 2

- Minimize the travel distance of the van, now without multiple sub tours.

New parameters

- C the number of customer and store locations.

New variables

- Counting variable $u_c \geq 0$. It will be required that if a customer is visited after another on the route then the later customer must have a u_c -value of at least one greater than the previously visited customer. Only the first customer is exempt from this rule.

Adjustments to the model

New constraints:

- The constraint which sets the counting variable for each customer:

$$u_{c1} + 1 \leq u_{c2} + C(1 - x_{c1,c2}) \quad \forall c1, c2 | c1 \neq c2 \wedge c2 \neq 1$$

The full model in Julia/JuMP, available with the name

Groceries2.jl

from the book web-site, is given below:

```
#####
# Groceries2, "Mathematical Programming Modelling" (42112)
#####
# Intro definitions
using JuMP
using HiGHS
#####

# PARAMETERS
C=6 # no of customers

#x and y- coordinates of all customers, c=0 is the store:
coord = zeros(C,2)
coord[1,1]=0;   coord[1,2]=0
coord[2,1]=104; coord[2,2]=19
coord[3,1]=370; coord[3,2]=305
coord[4,1]=651; coord[4,2]=221
coord[5,1]=112; coord[5,2]=121
coord[6,1]=134; coord[6,2]=515

Distance = zeros(Float64,C,C)
for c1 in 1:C
    for c2 in 1:C
        Distance[c1,c2]= sqrt( (coord[c1,1]-coord[c2,1])^2 + (coord[c1,2]-coord[c2,2])^2 )
    end
end
#####

#####
# Model
TSP = Model(HiGHS.Optimizer)

@variable(TSP, x[1:C,1:C], Bin)
for c in 1:C
    fix(x[c,c],0; force = true) #"Remove" nonsense variables by setting to 0
end
@variable(TSP, u[1:C]>=0)

# Minimize TSP distance
```

```

@Objective(TSP, Min,
           sum(Distance[c1,c2]*x[c1,c2] for c1=1:C, c2=1:C))

# Arrive at each customer ONCE:
@constraint(TSP, [c1=1:C],
           sum(x[c2,c1] for c2=1:C)==1)

# Leave each customer ONCE:
@constraint(TSP, [c1=1:C],
           sum(x[c1,c2] for c2=1:C)==1)

# Counter constraint:
@constraint(TSP, counter_con[c1=1:C,c2=2:C,c1!=c2],
           u[c1] + 1 <= u[c2] + C*(1-x[c1,c2])
           )

#####

#####
# solve
optimize!(TSP)
println("Termination status: $(termination_status(TSP))")
#####

#####
println("objective = $(objective_value(TSP))")
println("Solve time = $(solve_time(TSP))")
for c1=1:C
    for c2=1:C
        if value(x[c1,c2])>0.01
            println("Van drives from $c1 -> $c2")
        end
    end
end
end

println("Counters, u= ", round.(Int64,value.(u)))
#####

```

If you now look at the solution, it consists of a single tour visiting all customers: $c1 \rightarrow c2 \rightarrow c4 \rightarrow c3 \rightarrow c6 \rightarrow c5 \rightarrow c1$ (or the same tour in reverse). But the new tour is more expensive.

Notice how the restrictions on the new counting constraints have been implemented here. The condition that $c1 \neq c2$ is specified in Julia as $c1 \neq c2$ while the requirement that $c2 \neq 1$ was achieved by excluding 1 from the interval and instead setting $c2=2:C$.

When there are too many customers, with too large demands, several van's are

necessary. This complicates the problem significantly and creates the Vehicle Routing Problem (VRP). In this version of the problem, $K = 3$ vans need to be routed to customers such that: All customers get their groceries but without exceeding the vehicle capacity of 26. Hence we need three "TSP-like" routes, and the simple solution is: K routes leave the grocery store ($c = 1$), and K routes need to arrive back there. Finally, each route cannot exceed the capacity. Here we utilize the u_c counting variables, to count how much consumer goods the vans have transported (and delivered) until this customer. Thus, instead of adding 1 unit we add q_c unit. Finally, we change the domain of the u_c variables to be the capacity of the vans, hence ensuring that not too many customers are visited by a van. Another option could have been to keep the counting variables as they were and introduce a separate constraint for the capacity, but to do that we would need to know which customers are visited by what van, something this other method avoids.

Problem 3

- Minimize the travel of all the vans

New Parameters

- K : Number of vans, here 3
- Q : Van capacity, here 26

Changed decision variables

- Counting variable $0 \leq u_c \leq Q$. How much capacity has the van used for all the previous customers on its route when arriving at customer c ? When the van drives from a customer $c1$ to $c2$ then the u -variables increases by at least q_{c1} .

Model

Objective:

- Minimize the driven distance:

$$\sum_{c1,c2} Distance_{c1,c2} \cdot x_{c1,c2}$$

Constraints:

- You have to drive **to** each customer (except the grocery store) exactly once:

$$\sum_{c2} x_{c2,c1} = 1 \quad \forall c1 \mid c1 \neq 1$$

- You have to drive **away** from each customer (except the grocery store) exactly once:

$$\sum_{c2} x_{c1,c2} = 1 \quad \forall c1 \mid c1 \neq 1$$

- K vans have to drive **away** from the grocery store):

$$\sum_{c2} x_{1,c2} = K$$

- K vans have to drive **to** the grocery store:

$$\sum_{c2} x_{c2,1} = K$$

- The constraint which sets the counting variable for each customer:

$$u_{c1} + q_{c1} \leq u_{c2} + Q(1 - x_{c1,c2}) \quad \forall c1, c2 \mid c1 \neq c2 \wedge c2 \neq 1$$

The full model in Julia/JuMP, available with the name

`Groceries3.jl`

from the book web-site, is given below:

```

*****
# Vehicle Routing, "Mathematical Programming Modelling" (42112)
using JuMP
using HiGHS
*****

*****
# PARAMETERS
C=13 # no of customers

# customer coordinates and store:
coord = zeros(C,2)
coord[1,1]=0;   coord[1,2]=0
coord[2,1]=104; coord[2,2]=19
coord[3,1]=370; coord[3,2]=305
coord[4,1]=651; coord[4,2]=221

```



```

coord[5,1]=112; coord[5,2]=121
coord[6,1]=134; coord[6,2]=515
coord[7,1]=797; coord[7,2]=424;
coord[8,1]=347; coord[8,2]=444;
coord[9,1]=756; coord[9,2]=141;
coord[10,1]=304; coord[10,2]=351;
coord[11,1]=236; coord[11,2]=775;
coord[12,1]=687; coord[12,2]=310;
coord[13,1]=452; coord[13,2]=57;

# distance between customers
Distance = zeros(Float64,C,C)
for c1 in 1:C
    for c2 in 1:C
        Distance[c1,c2]= sqrt( (coord[c1,1]-coord[c2,1])^2 + (coord[c1,2]-coord[c2,2])^2)
    end
end

# size of deliveries
q = zeros(C)
q[1]=0;
q[2]=3;
q[3]=9;
q[4]=7;
q[5]=11;
q[6]=11;
q[7]=6;
q[8]=7;
q[9]=7;
q[10]=2;
q[11]=4;
q[12]=2;
q[13]=8;

# number of vans
K=3

# capacity of each of the vans
Q=26
#####

#####

# Model
VRP = Model(HiGHS.Optimizer)

@variable(VRP, x[1:C,1:C], Bin)

```

```

for c in 1:C
    fix(x[c,c],0; force = true) "Remove" nonsense variables by setting to 0
end

@variable(VRP, 0 <= u[1:C] <= Q)

# Minimize VRP distance
@objective(VRP, Min,
    sum(Distance[c1,c2]*x[c1,c2] for c1=1:C, c2=1:C))

# every customer is entered by one van, except the store on node 1
@constraint(VRP, [c1=2:C],
    sum(x[c2,c1] for c2=1:C) == 1)

# every customer is exited by one van, except the store on node 1
@constraint(VRP, [c1=2:C],
    sum(x[c1,c2] for c2=1:C) == 1)

# K vans going out of node 1
@constraint(VRP,
    sum(x[1,c2] for c2=2:C) == K)

# K trucks going in of node 1
@constraint(VRP,
    sum(x[c2,1] for c2=2:C) == K)

# Combined counter and capacity constraint
@constraint(VRP, [c1=2:C,c2=1:C,c1!=c2],
    u[c1] + q[c1] <= u[c2] + Q*(1-x[c1,c2]) )

#####

#####
# solve
optimize!(VRP)
println("Termination status: $(termination_status(VRP))")
#####

#####
#print(VRP)
println("objective = $(objective_value(VRP))")
println("Solve time: $(solve_time(VRP))")
#####

```

Notice: The way that capacity is tracked and constrained in this model does not rely on us knowing exactly which van visits what customer. This helps keep the

model small and free of symmetry issues compared to other modeling choices.